

SPIDER-MAN RESCUE

Submitted By

Student Name	Student ID
Affan Julfiker	232-15-682
Samiul Haque	232-15-098

LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course
**CSE412/422: Computer Graphics Lab in the Department of Computer
Science and Engineering**



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

13/08/2026

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of **Faiza Feroz, Lecturer**, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Faiza Feroz

Lecturer

Department of Computer Science and Engineering
Daffodil International University

Submitted by

Affan Julfiker ID: 232-15-682 Dept. of CSE, DIU	Samiul Haque ID: 232-15-098 Dept. of CSE, DIU
--	--

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO1	Design and implement fundamental graphics primitives and algorithms to construct a 2D urban rescue scene
CO2	Apply OpenGL programming, geometric transformations, line/circle algorithms, clipping, and animation techniques.
CO3	Develop, document, and demonstrate a complete computer graphics project using modular design and structured implementation.
CO4	Present and explain the project, its engineering activities, algorithms, transformations, animation logic, and results through a technical report and demonstration.

Table 2: Mapping of CO, PO, Blooms, KP and CEP

Mapping Course Outcome (COs) with the Teaching-Learning and Assessment Strategy:

The mapping justification of this table is provided in section 4.2.1, 4.2.2 and 4.2.3.

CO	POs	Teaching Activities	Learning Activities	Assessment Strategy	Learning Domains	Knowledge Profile	Complex Engineering Problem	Complex Engineering Activities
CO1	PO1	TLA1 (Lab exercises)		Lab performance, Lab report	C2	K1, K3	EP1, EP4	
CO2	PO5	TLA2 (Algorithm implementation)		Lab performance, Lab report	C3, C4	K6	EP1, EP4, EP3	
CO3	PO9	TLA3 (Group project)		Lab project	P3, A2			
CO4	PO10	TLA4 (Presentation & Viva)		Project & Viva	C6, P3, A2			EA1, EA3

Table of Contents

Declaration	2
Course & Program Outcome	2
1 Introduction	5
1.1 Introduction	5
1.2 Motivation	5
1.3 Objectives	5
1.4 Feasibility Study	6
1.5 Gap Analysis	6
1.6 Project Outcome	6
2 System Architecture / Scene Description & Architecture	7
2.1 Requirement Analysis & Design Specification	7
2.1.1 Requirements	7
2.1.2 Sketch of project	8
2.2 Use of Modern Tools	9
2.3 Algorithm Used	9
2.4 2D Transformations Applied	9
2.5 Animation Logic	9
3 Implementation and Results	11
3.1 Implementation	11
3.2 Output	11
3.3 Discussion	12
4 Engineering Standards and Mapping	13
4.1 Impact on Society, Environment and Sustainability	13
4.1.1 Impact on Life	13
4.1.2 Impact on Society & Environment	13
4.1.3 Ethical Aspects	13
4.1.4 Sustainability Plan	14
4.2 Complex Engineering Problem	14
4.2.1 Mapping of Program Outcome	14
4.2.2 Complex Problem Solving	14
4.2.3 Engineering Activities	15

5 Conclusion	16
5.1 Summary	16
5.2 Limitation	16
5.3 Future Work	16
References	17

Chapter 1

Introduction

This chapter presents the background, motivation, objectives, feasibility, development gap, and expected outcome of the Spider-Man Rescue project. The project is an interactive 2D urban rescue simulation designed to demonstrate computer graphics concepts through a continuous animated scene.

1.1 Introduction

Computer graphics is the field of creating, manipulating, and displaying visual information using computational methods. OpenGL provides a programming interface for rendering graphical primitives and building interactive visual applications. In this project, a 2D orthographic world is used to construct an urban environment containing buildings, roads, vehicles, pedestrians, trees, street lamps, clouds, birds, a distant aircraft, Spider-Man, and a citizen who requires rescue.

The central interaction is a rescue scenario. A citizen begins on a rooftop while a countdown runs. If the user presses S before or during the fall, Spider-Man is launched using a web and continuously re-aims toward the citizen's current position. The simulation can end in either a successful rescue or a mission failure. Background elements continue to animate throughout the scene.

1.2 Motivation

The project was motivated by the need to combine multiple Computer Graphics Lab concepts into one visually engaging and interactive application rather than demonstrating each algorithm separately. A rescue simulation provides a clear story in which line drawing, circle generation, transformations, collision detection, animation timing, clipping, lighting transitions, and keyboard interaction can work together.

The project also provides a practical demonstration of how mathematical graphics techniques can be used to create dynamic characters and environments without depending on imported 2D artwork for the main scene.

1.3 Objectives

- Implement fundamental graphics primitives including DDA line drawing, Bresenham line drawing, and the Midpoint Circle algorithm.
- Implement a cubic Bezier curve for the visual web line and Cohen-Sutherland clipping to keep selected line segments within the viewport.
- Apply translation, rotation, and scaling transformations to moving and animated objects.
- Create an interactive 2D urban rescue scenario using C++ and OpenGL/GLUT.
- Implement real-time animation using a fixed 30 ms simulation timestep.
- Design a GameState-based rescue sequence with waiting, falling, rescue, carrying, talking, walking, success, and failure states.

- Demonstrate collision detection between Spider-Man and the falling citizen.
- Provide day/night lighting transitions, HUD information, keyboard controls, pause, restart, and debugging support.

1.4 Feasibility Study

The project is technically feasible because it uses a conventional C++17/OpenGL/GLUT architecture and a single source file. The supplied implementation uses standard GLUT entry points and does not require GLEW. The project can be built with a command similar to: `g++ -std=c++17 main.cpp -o main -lglut -lGLU -lGL -lm && ./main`.

From a methodological perspective, the project combines established computer graphics algorithms with an application-level state machine. This makes the system suitable for a laboratory environment where students need to demonstrate both algorithmic understanding and practical OpenGL programming.

1.5 Gap Analysis

Many basic Computer Graphics Lab projects demonstrate a single scene or one isolated algorithm, such as drawing a line, circle, house, or traffic object. The present project addresses the gap between isolated demonstrations and an integrated interactive application. It combines multiple manual algorithms, transformations, animation, interaction, clipping, collision detection, and lighting in a single scenario.

Another gap is the lack of meaningful interaction in static graphical scenes. Here, the user has a time-sensitive decision through the S key, while the result is determined by the rescue state and collision condition. Therefore, the project provides a more application-oriented demonstration of the course concepts.

1.6 Project Outcome

The final outcome is an interactive urban rescue simulation rendered in a 1150×720 world-space layout. The scene continuously animates traffic, pedestrians, clouds, birds, an aircraft, lighting, and other environmental elements while the rescue sequence is controlled by the user. A successful rescue ends with the citizen safely delivered to the walkway; a failed attempt produces a non-graphic particle effect and a visible failure state. The source code explicitly describes the project as a C++/OpenGL/GLUT Computer Graphics Lab showcase project.

Chapter 2

System Architecture/ Scene Description & Architecture

The project uses a structured single-file C++ architecture divided into logical sections: global constants, enumerations, basic structures, graphics primitives and algorithms, 2D transformations, lighting, initialization, environment drawing, actors, impact particles, HUD, game logic, keyboard handling, GLUT callbacks, restart/diagnostics, and main.

2.1 Requirement Analysis & Design Specification

2.1.1 Requirements

The following functional and technical requirements are derived from the submitted implementation:

- C++17-compatible compiler and OpenGL/GLUT or FreeGLUT environment.
- 2D orthographic rendering using a world-space coordinate system.
- Manual implementation of DDA, Bresenham, Midpoint Circle, Bezier curve, and Cohen-Sutherland clipping.
- Data structures for buildings, vehicles, pedestrians, trees, lamps, clouds, particles, birds, and points.
- Interactive keyboard input for rescue, lighting, pause, debugging, help, restart, and quit.
- Fixed-timestep animation and continuous environment updates.
- Game-state management for the complete rescue story.
- Collision detection and physics-inspired falling behavior.
- HUD and diagnostics for the live state, elapsed time, lighting, particle count, and controls.

2.1.2 Sketch of project



2.2 Use of Modern Tools

- C++17 – primary programming language for the graphics, animation, state management, and input logic.
- OpenGL – used for rendering points, lines, rectangles, circles, and other graphical components.
- GLUT / FreeGLUT – used for window management, keyboard callbacks, timer callbacks, and bitmap text.
- GLU – used for the 2D orthographic projection through the GLUT/OpenGL setup.
- Standard C++ libraries – `cmath`, `cstdio`, `cstdlib`, `string`, `vector`, and algorithm support mathematical calculations and data management.
- Git/GitHub – suitable for source-code version control and collaborative development, although the supplied source itself only establishes the application architecture and build requirements.

2.3 Algorithm Used

The implementation intentionally uses manual graphics algorithms. DDA is used for thin lines such as window mullions, lane markings, and aircraft contrail segments. Bresenham's algorithm is used for figure limbs, web strand accents, edges, bird strokes, and selected tree/roof details. The Midpoint Circle algorithm is used for heads, vehicle wheels, the sun/moon, stars, and tree canopies.

A cubic Bezier curve is used to create the sagging appearance of the web during Spider-Man's swing. Cohen-Sutherland line clipping is applied before drawing the web/debug line so that a line segment that extends beyond the viewport is clipped to the visible region.

2.4 2D Transformations Applied

Three reusable transformation functions are defined: `translatePoint`, `scalePoint`, and `rotatePoint`. Translation is used for traffic, pedestrians, clouds, birds, the aircraft, the falling citizen, the carrying motion, and Spider-Man's exit. Rotation is used for the homing swing and pedestrian leg animation. Scaling is available around an arbitrary pivot and is also used for size adjustments and vertical mirroring/reflection effects.

The Spider-Man swing is particularly dependent on rotation. The program computes a target angle and target web length from the web anchor to the citizen's current position, then adjusts the swing parameters and rotates Spider-Man around the anchor.

2.5 Animation Logic

Animation is controlled by a fixed timestep of 0.03 seconds and a 30 ms GLUT timer. During each update, the program advances environment motion, particles, lighting, the sun/moon angle, and the active rescue state. The fixed timestep makes movement more predictable than tying motion directly to an uncontrolled rendering frame rate.

The citizen's fall uses gravity of 260 pixels per second squared. The velocity is updated each tick and the position is then translated downward. Spider-Man's rescue motion continuously recalculates the target angle and length based on the citizen's live position, making the swing an actively guided movement rather than a fixed-radius pendulum.

Chapter 3

Implementation and Results

This chapter explains how the project was implemented and how the main components work together to produce the final interactive scene.

3.1 Implementation

The implementation begins with global scene dimensions and simulation constants. The initial window is 1150×720 and the scene uses an orthographic 2D coordinate system. Scene entities are represented by lightweight structures such as Point2D, Building, Vehicle, Pedestrian, Tree, StreetLamp, Cloud, Particle, and Bird.

The rendering pipeline follows a back-to-front approach. The sky and celestial objects are drawn first, followed by background scenery, road and sidewalk elements, actors, and finally the HUD. A short camera-shake offset is applied during failure feedback.

The day/night system uses two palettes and interpolates between them. The sky colors, building shade, window lighting chance, lamp state, sun visibility, and moon visibility are derived from the current interpolated palette. The user can request Day or Night using D or N.

The rescue logic is controlled by the GameState enumeration: WAITING, JUMPING, RESCUE, CARRY, TALK, WALK, SUCCESS, and FAILURE. The S key is active during WAITING and JUMPING. If rescue is initiated, Spider-Man is connected to the web anchor and guided toward the falling citizen. A distance threshold determines the catch.

3.2 Output

The expected visual output is a continuously animated urban scene containing a varied skyline, road traffic, pedestrians, trees, street lamps, clouds, birds, and an aircraft. The central story shows a citizen on a rooftop and Spider-Man on another building.

During a successful run, Spider-Man catches the citizen, carries the citizen to the landing area, pauses for a short dialogue beat, and then the citizen walks along the sidewalk while Spider-Man exits the scene. The terminal state becomes MISSION SUCCESS.

During a failed run, the citizen reaches the ground, a brief particle burst is shown, a subtle camera shake is applied, and the terminal state becomes MISSION FAILED. The scene remains visible rather than removing the citizen.

The HUD provides the current state, elapsed simulated time, lighting mode, particle count, and a controls/help panel. The debug grid can be toggled with G, and the simulation can be paused with P.

3.3 Discussion

The project successfully integrates low-level graphics algorithms with higher-level application logic. One important design feature is the separation between rendering and simulation updates. The `updateGame` function advances the environment and then executes the appropriate state-specific rescue behavior.

The rescue interaction also demonstrates why transformations and collision detection are important in computer graphics. The citizen's position changes continuously under gravity, while Spider-Man's position is calculated relative to a web anchor. The collision test then determines whether the two stylized figures are sufficiently close for a successful catch.

The project therefore demonstrates more than static rendering: it combines geometry, rasterization algorithms, transformations, time-based animation, interaction, clipping, lighting interpolation, and state management into one coherent application.

Chapter 4

Engineering Standards and Mapping

This chapter maps the project activities to the course and program outcomes and discusses social, environmental, ethical, and sustainability considerations in line with the supplied report template.

4.1 Impact on Society, Environment and Sustainability

4.1.1 Impact on Life

The project is an educational simulation rather than a real emergency-response system. Its main impact on life is therefore learning-oriented: it helps students understand how graphical systems can represent movement, interaction, timing, and visual feedback. The rescue scenario also provides an intuitive context for demonstrating these technical concepts.

4.1.2 Impact on Society & Environment

As a software-based academic project, the simulation has a small physical environmental footprint compared with hardware-based systems. It can be used as a classroom demonstration of interactive graphics, visualization, and animation without requiring specialized physical equipment.

The urban environment visually represents common city elements such as roads, buildings, pedestrians, vehicles, trees, and street lamps. This makes the project relatable while demonstrating how a graphical scene can model a simplified real-world setting.

4.1.3 Ethical Aspects

The project should be presented as a fictional, non-graphic rescue simulation. The implementation intentionally uses a brief particle effect and camera shake for failure feedback instead of graphic injury representation. The project documentation should also maintain academic integrity by clearly acknowledging learning resources and avoiding unauthorized use of external assets.

4.1.4 Sustainability Plan

The software can be maintained by keeping the graphics algorithms, transformations, animation logic, and state-management functions modular. Future contributors can modify individual scene components without redesigning the entire application. The use of standard C++ and GLUT-compatible entry points also supports portability to systems with suitable OpenGL/GLUT support.

4.2 Complex Engineering Problem

The project involves a complex engineering problem because multiple graphical requirements must operate simultaneously: rasterization algorithms must create the visual primitives, transformations must update moving objects, animation must remain time-consistent, the rescue state machine must control the story, and collision detection must determine the outcome.

4.2.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO	Justification of Mapping (Project Perspective)
PO1	The project applies mathematical and computational knowledge to coordinates, geometry, line/circle generation, transformations, motion, and collision calculations.
PO5	The project uses modern engineering tools including C++, OpenGL, GLUT/FreeGLUT, and standard libraries to implement and demonstrate the graphical system.
PO9	The project can be developed through task allocation among team members covering graphics algorithms, scene design, animation, testing, documentation, and presentation.
PO10	The project requires a technical report and live demonstration in which the algorithms, transformations, architecture, and results are communicated clearly.

4.2.2 Complex Problem Solving

The following table presents the mapping of the project with the **Complex Engineering Problem (CEP) attributes defined by the Washington Accord framework**. The attributes that are addressed in this project are marked (✓).

Table 4.2: Mapping with complex problem solving.

EP1 Depth of Knowledge	EP2 Range of Conflicting Requirements	EP3 Depth of Analysis	EP4 Familiarity of Issues	EP5 Extent of Applicable Codes	EP6 Extent Of Stakeholder Involvement	EP7 Inter- dependence
------------------------------	--	-----------------------------	---------------------------------	---	---	-----------------------------

✓		✓	✓			
---	--	---	---	--	--	--

CEP Attribute	Project Justification
EP1 – Depth of Knowledge	The project combines rasterization algorithms, geometry, transformations, animation, clipping, collision detection, and OpenGL rendering.
EP3 – Depth of Analysis	The rescue behavior requires analysis of positions, angles, distances, velocity, timing, and state transitions.
EP4 – Familiarity of Issues	The project applies familiar Computer Graphics concepts such as DDA, Bresenham, Midpoint Circle, Bezier curves, transformations, and GLUT callbacks.

4.2.3 Complex Engineering Activities

Table 4.3 presents the mapping of the project with the **Complex Engineering Activities (CEA) attributes**. The attributes relevant to this project are marked (✓).

Table 4.3: Mapping with complex engineering activities.

EA1	EA2	EA3	EA4	EA5
✓		✓		

CEA	Project Justification
EA1	The project follows a systematic design process for scene entities, graphics algorithms, transformations, state management, rendering order, and documentation.
EA3	The project demonstrates innovation through an interactive rescue scenario that combines multiple graphical elements, real-time animation, user input, day/night transitions, and visual feedback.

Chapter 5

Conclusion

5.1 Summary

Spider-Man Rescue is an interactive 2D Computer Graphics Lab project implemented in C++ using OpenGL/GLUT. It creates a complete urban environment and integrates manual graphics algorithms, transformations, animation, interaction, collision detection, clipping, lighting transitions, and a state-based rescue sequence.

The project demonstrates DDA, Bresenham, Midpoint Circle, Bezier curve, and Cohen-Sutherland clipping. Translation, rotation, and scaling are implemented as reusable mathematical operations. A fixed 30 ms timestep controls the animation, while the GameState machine controls the rescue narrative.

5.2 Limitation

- The scene is a stylized 2D simulation rather than a physically accurate urban or rescue environment.
- Collision detection between Spider-Man and the citizen uses a simple distance threshold rather than detailed shape intersection.
- The physics model is intentionally simplified for real-time graphical demonstration.
- The implementation is concentrated in a single source file, which can become harder to maintain as the project grows.
- The current project does not provide a persistent score, multiple rescue levels, or external data storage.

5.3 Future Work

- Add multiple rescue scenarios and increasing difficulty levels.
- Introduce a score, timer-based performance rating, and rescue statistics.
- Improve character animation with more detailed poses and smoother skeletal motion.
- Add richer environmental effects such as rain, fog, reflections, and more detailed lighting.
- Separate the single source file into modules for algorithms, rendering, entities, input, and game logic.
- Add sound effects and background music while keeping the project lightweight.
- Add more precise collision models and optional physics-based web movement.

- Provide a cross-platform build configuration and automated project documentation.

References

1. OpenGL Programming Guide, Addison-Wesley.
2. GeeksforGeeks, OpenGL tutorials and computer graphics algorithm resources.
3. Stack Overflow discussions and community resources related to OpenGL animation and GLUT programming.
4. Submitted project source code: “Spider-Man Rescue: Interactive Urban Rescue Simulation,” C++ / OpenGL / GLUT implementation.
5. Supplied Computer Graphics Lab Project Report template, Daffodil International University.